

Database Systems Overview

Relational Databases

Relational databases store data in structured tables with predefined schemas and support SQL queries. They enforce **ACID** (Atomicity, Consistency, Isolation, Durability) properties to guarantee data integrity [Codd, 1970].

Database	Released	License	ACID Support	Primary Use Case
PostgreSQL	1996	PostgreSQL (BSD)	Full	General-purpose OLTP
MySQL	1995	GPL/Commercial	Full	Web applications
SQLite	2000	Public Domain	Full	Embedded systems
Oracle DB	1979	Commercial	Full	Enterprise OLTP
MariaDB	2009	GPL	Full	Web applications

ACID Properties

- **Atomicity:** All operations in a transaction succeed or all are rolled back
 - Prevents partial updates that leave data in an inconsistent state
 - Implemented via write-ahead logs (WAL) in most modern databases
- **Consistency:** Data must satisfy all predefined rules and constraints
 - Includes foreign key constraints, unique constraints, and check constraints
 - Enforced before and after every transaction
- **Isolation:** Concurrent transactions do not interfere with each other
 - Levels: Read Uncommitted → Read Committed → Repeatable Read → Serializable
 - PostgreSQL defaults to Read Committed isolation
- **Durability:** Committed transactions survive system crashes
 - Achieved through write-ahead logging and periodic checkpoints

NoSQL Databases

NoSQL databases sacrifice strict ACID guarantees for horizontal scalability and flexible schemas [Cattell, 2011].

Database	Type	Consistency Model	Query Language	Best For
MongoDB	Document	Tunable	MQL	Semi-structured data
Cassandra	Wide-column	Eventual	CQL	Time-series, IoT
Redis	Key-Value	Strong (single)	Redis commands	Caching, sessions
Neo4j	Graph	ACID	Cypher	Relationship traversal
Elasticsearch	Search/Document	Eventual	Query DSL	Full-text search

CAP Theorem

According to the **CAP theorem** (Brewer, 2000), a distributed system can guarantee at most two of the following three properties:

1. **Consistency** — Every read returns the most recent write
2. **Availability** — Every request receives a response
3. **Partition Tolerance** — System operates despite network partitions

"You can only pick two out of three, and since partition tolerance is essential in distributed systems, the real choice is between Consistency and Availability." — Eric Brewer, 2000

Choosing a Database

Decision factors, ordered by importance:

1. Data model fit
 - Structured relational data → PostgreSQL or MySQL
 - Hierarchical or variable schema → MongoDB
 - Graph relationships → Neo4j
 - Time-series events → Cassandra or InfluxDB
2. Consistency requirements
 - Financial transactions requiring full ACID → PostgreSQL, Oracle
 - High-availability reads with eventual consistency acceptable → Cassandra
3. Scale requirements
 - Single-node up to ~10 TB → PostgreSQL with read replicas
 - Multi-region horizontal scale → Cassandra, DynamoDB

References

- [Codd, 1970] E.F. Codd, "A Relational Model of Data for Large Shared Data Banks," *CACM*, 1970.
- [Cattell, 2011] R. Cattell, "Scalable SQL and NoSQL Data Stores," *SIGMOD Record*, 2011.